

Lecture 4:

Scoring, term weighting and the vector space model

Today's Lecture

Ranked retrieval

Scoring documents

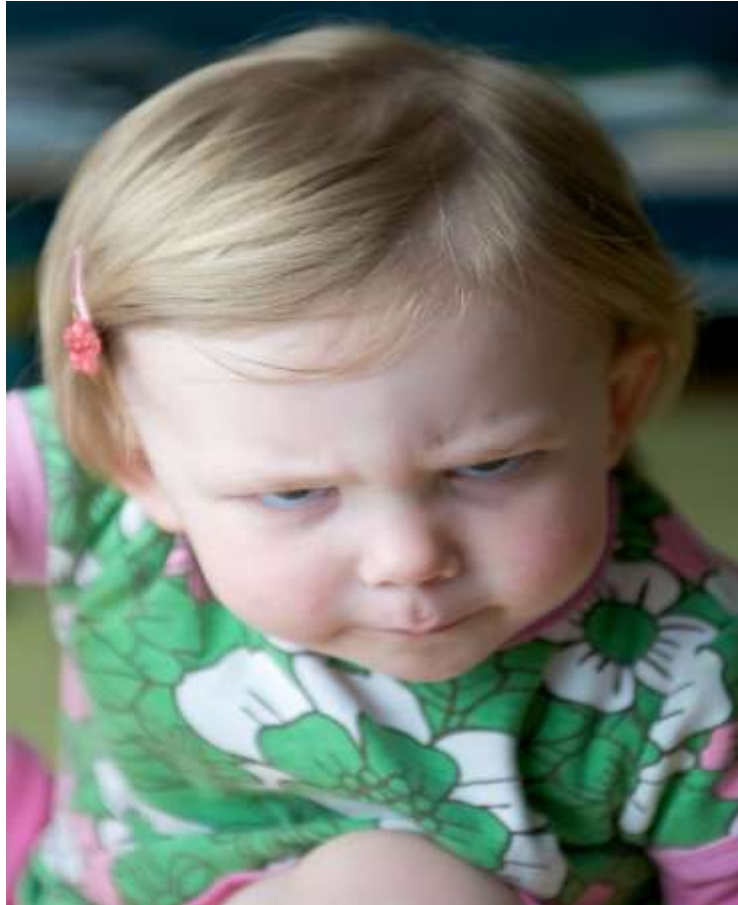
Term frequency

Collection statistics

Weighting schemes

Vector space scoring

Boolean Queries



Remember the
Boolean Retrieval?

Boolean Queries

Thus far, our queries have all been Boolean.

- Documents either match or don't.
- Good for expert users with precise understanding of their needs and the collection
- Also good for applications: Applications can easily consume 1000s of results.

Not good for the majority of users.

- Most users incapable of writing Boolean queries (or they are, but they think it's too much work).
- Most users don't want to wade through 1000s of results.
 - This is particularly true of web search.

Problems with Boolean Search

- Boolean queries often result in either too few(=0) or too many (1000s) results
- Query 1 :“standard user drink 650” : 200000 hits
- Query 2 :“standard user drink 650 no card found” : 0 hits
- It takes a lot of skills to come up with a query that produces a manageable number of hits
 - AND gives too few :OR gives too many

Ranked retrieval models

- Rather than a set of documents satisfying a query expression, in ranked retrieval, the system returns an ordering over the (top) documents in the collection for a query
- Free text queries : Rather than a query language of operators and expressions, the user's query is just one or more words in a human language
- In principle, there are two separate choices here, but in practice, ranked has normally been associated with free text queries and vice versa

Ranked Retrieval - Cont..

- When a system produces a ranked result set, large result sets are not an issue
 - Indeed, the size of the result set is not an issue
 - We just show the top k (≈ 10) results
 - We don't overwhelm the user
- Premise: the ranking algorithm works

How to rank a document given a query

- We need a scoring mechanism
 - $\text{Score}(Q, D_i)$
- Any scoring you could suggest ?
 - Query of few words
 - Documents of varying size of words
 - Comparable
 - Preformat
- We wish to return in order the documents most likely to be useful to the searcher
- How can we rank-order the documents in the collection with respect to a query?
 - Assign a score - say in $[0, 1]$ - to each document
- This score measures how well document and query “match”.

Query- document matching scores

- We need a way of assigning a score to a query/document pair
- Let's start with a one-term query
- If the query term does not occur in the document: score should be 0
- The more frequent the query term in the document, the higher the score(should be)
- We will look at a number of alternatives for this

Option 1: Jaccard coefficient

$$Jaccard(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

$$jaccard(A, A) = 1$$

$$jaccard(A, B) = 0 \rightarrow \text{no overlap}$$

Size of the A & B are not important

Normalized

- $[0, 1]$

Example

What is the query-document match score that the Jaccard coefficient computes for each of the two documents below?

- Query: *ides of march*
- Document1: *caesar died in march*
- Document2: *the long march*
- $\text{jaccard}(Q, D) = |Q \cap D| / |Q \cup D|$
 - $\text{jaccard}(\text{Query}, \text{Document1}) = 1/6$
 - $\text{jaccard}(\text{Query}, \text{Document2}) = 1/5$

Drawbacks of using Jaccard

- It does not consider term frequency(How many times a term occur in a document)
- Rare Items in a collection are more informative than frequent terms. Jaccard doesn't consider this information
- We need a more sophisticated way of normalizing for length

Jaccard with Bag of words Model

- Vector representation doesn't consider the ordering of words in a document

Same vector for

- *John is quicker than Mary*
- *Mary is quicker than John*

The positional index was able to distinguish these two documents

- There are methods for “recovering” positional information

Term frequency tf

- The term frequency $tf_{t,d}$ of term t in document d is defined as the number of times that t occurs in d .
- We want to use tf when computing query-document match scores.
 - A document with 10 occurrences of the term is more relevant than a document with 1 occurrence of the term.
 - But not 10 times more relevant
 - Not linear
 - Relevance does not increase proportionally with term frequency.

Binary term-document incidence matrix

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Each document is represented by a binary vector $\in \{0, 1\}^{|v|}$

Term-document count matrices

- Consider the number of occurrences of a term in a document:
- Each document is a count vector in $\mathbb{N}^V$: a column below

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	157	73	0	0	0	0
Brutus	4	157	0	1	0	0
Caesar	232	227	0	2	1	1
Calpurnia	0	10	0	0	0	0
Cleopatra	57	0	0	0	0	0
mercy	2	0	3	5	5	1
worser	2	0	1	1	1	0

Log-frequency weighting

- The log frequency weight of term t in d is

$$w_{t,d} = \begin{cases} 1 + \log_{10} tf_{t,d} & tf_{t,d} > 0 \\ 0 & otherwise \end{cases}$$

- $0 \rightarrow 0, 1 \rightarrow 1, 2 \rightarrow 1.3, 10 \rightarrow 2, 1000 \rightarrow 4$, etc.
- The score is 0 if none of the query terms is present in the document.
- Score for a document-query pair: sum over terms t in both q and d :

$$Score = \sum_{t \in q \cap d} (1 + \log tf_{t,d})$$

Document frequency (df)

- Rare terms are more informative than frequent terms
 - Recall stopwords
- Consider a term in the query that is rare in the collection(eg : arachnocentric)
- A document containing this term is very likely to be relevant to the query arachnocentric
- We want a high weight for rare terms like arachnocentric

Idf weight

- df_t is the document frequency of t :
 - the number of documents that contain t
 - inverse measure of the informativeness of t
 - Max bound N
- We define the idf (inverse document frequency) of t by

$$idf_t = \log_{10} \left(\frac{N}{df_t} \right)$$

- We use log to “dampen” the effect of idf

Example

term	df_t	idf_t
Pythagoras	1	
thermo	10	
higher	100	
system	1,000	
the	10,000	

- Idf has no effect on ranking one term queries
- There is one idf value for each term t in a collection.

Example

term	df_t	idf_t
Pythagoras	1	$\log 10,000$
thermo	10	$\log 1000$
higher	100	$\log 100$
system	1,000	$\log 10$
the	10,000	$\log 1$

- Idf has no effect on ranking one term queries
- There is one idf value for each term t in a collection.

Collection Vs. document frequency

- Collection frequency of term t is the number of occurrences of t in the collection
- Document frequency of t is the number of documents in which t occurs

Why D F rather than C F

Word	Collection frequency	Document frequency
<i>insurance</i>	10440	3997
<i>try</i>	10422	8760

tf-idf

- The tf-idf weight of a term is the product of its tf weight and its idf weight

$$W_{t,d} = (1 + \log_{10}(tf_{t,d})) * \log_{10}(N/df_t)$$

- Best known weighting scheme in information retrieval
- Increases with the number of occurrences within a document
- Increases with the rarity of the term in the collection

Score for a document given a query

$$Score(q, d) = \sum_{t \in q \cap d} tf * idf_{t,d}$$

Binary -> Count -> Weight

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	5.25	3.18	0	0	0	0.35
Brutus	1.21	6.1	0	1	0	0
Caesar	8.59	2.54	0	1.51	0.25	0
Calpurnia	0	1.54	0	0	0	0
Cleopatra	2.85	0	0	0	0	0
mercy	1.51	0	1.9	0.12	5.25	0.88
worser	1.37	0	0.11	4.15	0.25	1.95

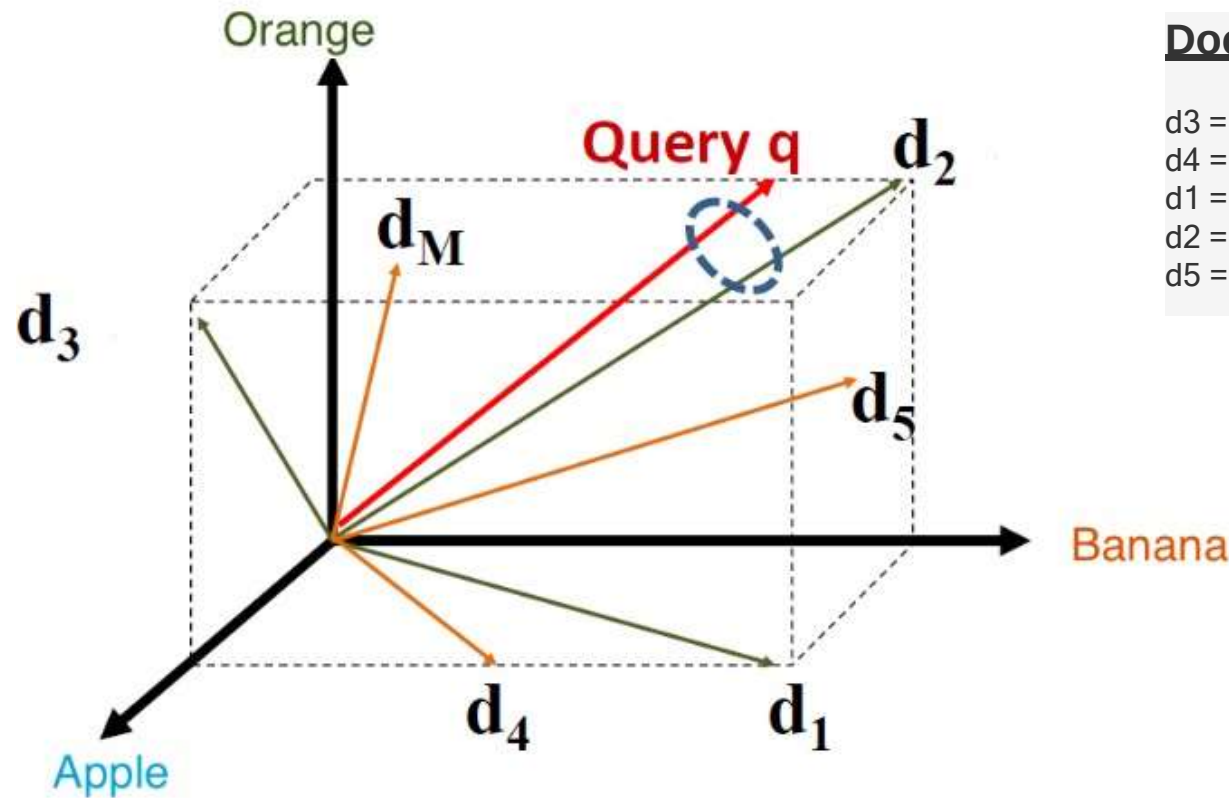
Documents as vectors

- We have a $|V|$ -dimensional vector space
 - Document is a point in that space represented by a vector
- Very high-dimensional: tens of millions of dimensions when you apply this to a web search engine
- These are very sparse vectors -most entries are zero

Queries as Vectors

- Represent Queries as vectors in the space
- Rank documents according to their proximity to the query in this space
- Proximity = similarity of vectors
- How is this different from Boolean model
 - Ranked retrieval

Queries as Vectors



Documents

d_3 = "Apple Orange Orange Apple"

d_4 = "Apple Banana Apple Banana"

d_1 = "Banana Apple Banana Banana Banana Apple"

d_2 = "Banana Orange Banana Banana Orange Banana"

d_5 = "Banana Apple Banana Banana Orange Banana"

Word/Term

Apple ==>> [1,0,0]

Banana ==>> [0,1,0]

Orange ==>> [0,0,1]

Drawbacks of using Jaccard

- It does not consider term frequency(How many times a term occur in a document)
- Rare Items in a collection are more informative than frequent terms. Jaccard doesn't consider this information
- We need a more sophisticated way of normalizing for length

Euclidean distance

- First cut: distance between two points
 - (= distance between the end points of the two vectors)

- Euclidean distance?

$$D(a, b) = \sqrt{\sum_k (a_k - b_k)^2}$$

- Euclidean distance is a bad idea ...
- ...because Euclidean distance is large for vectors of different lengths.

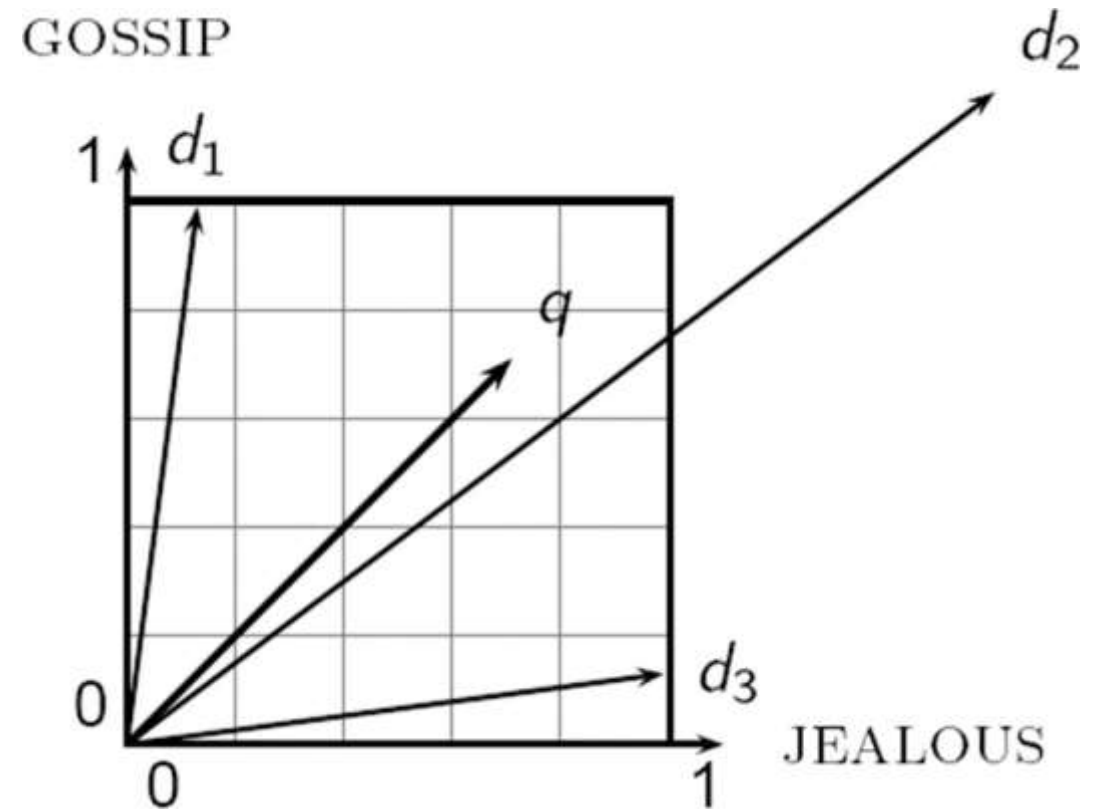
Use angle instead of distance

- Thought experiment:
 - take a document d
 - append it to itself , called d'
 - “Semantically” d and d' have the same content
- The Euclidean distance between the two documents can be quite large
- The angle between the two documents is 0
 - corresponding to maximal similarity

Key idea: Rank documents according to angle with query.

Why Distance is a bad idea

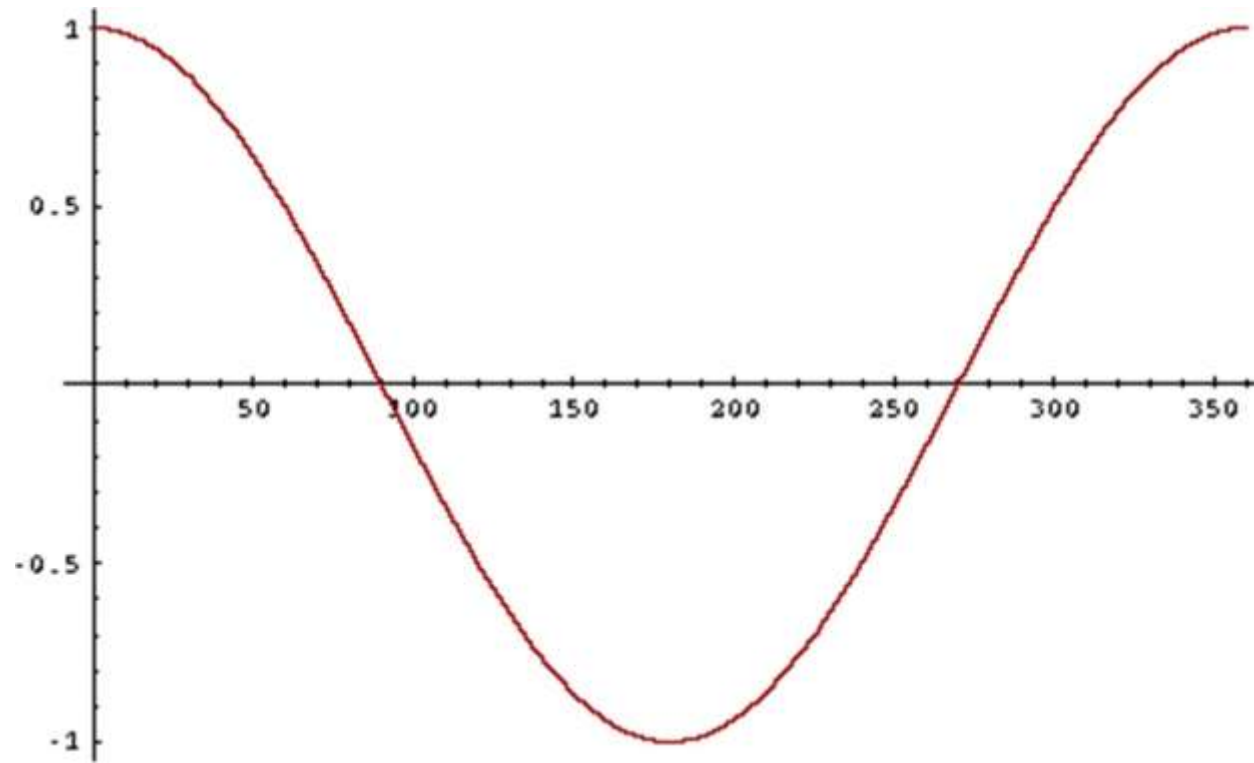
The Euclidean distance between q and d_2 is large even though the distribution of terms in the query q and the distribution of terms in the document d_2 are very similar.



Cosine(query, document)

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

From Angles to Cosines

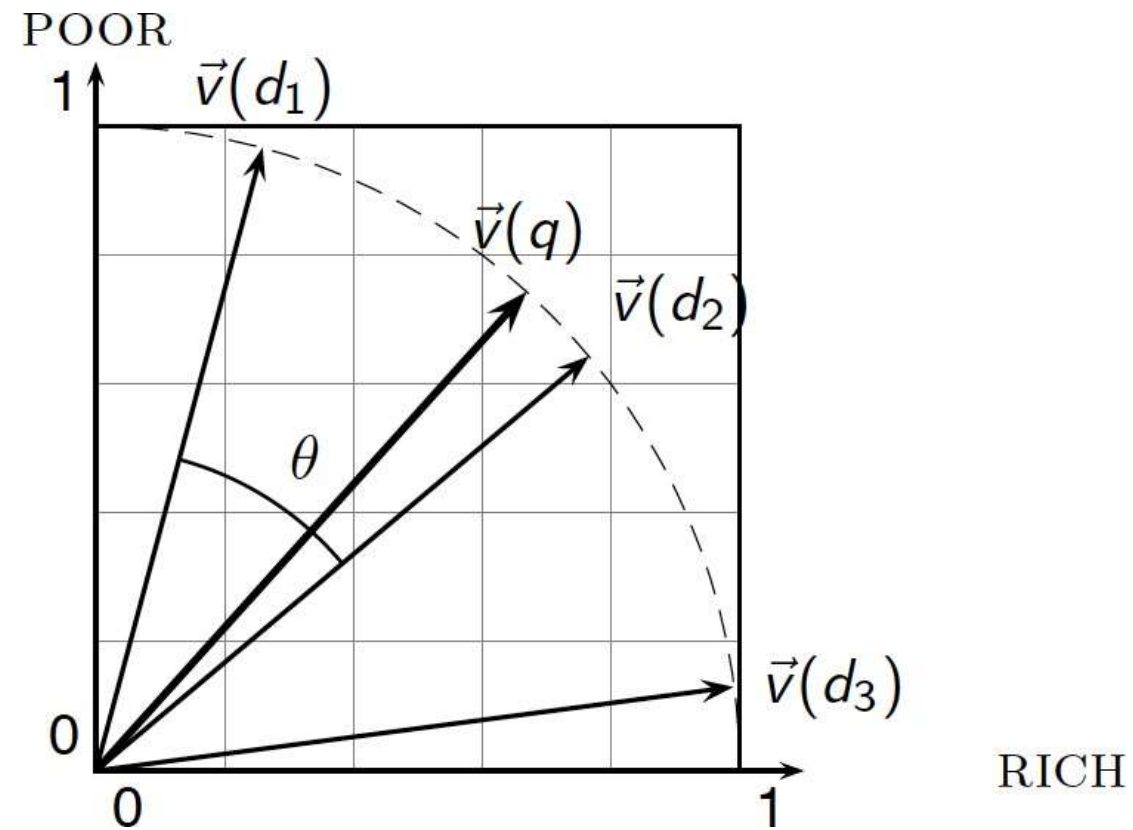


Cosine for length-normalized vectors

- For length-normalized vectors, cosine similarity is simply the dot product (or scalar product)

$$\cos(\vec{q}, \vec{d}) = \vec{q} \bullet \vec{d} = \sum_{i=1}^{|V|} q_i d_i$$

Cosine similarity illustrated



Cosine Similarity Example

How similar are the novels

- SaS: *Sense and Sensibility*
- PaP: *Pride and Prejudice*, and
- WH: *Wuthering Heights*

term	S _a S	P _a P	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Term frequencies (counts)

Cosine Similarity Example

Log frequency weighting

term	S _a S	P _a P	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

After length normalization

term	S _a S	P _a P	WH
affection	0.789	0.832	0.524
jealous	0.515	0.555	0.465
gossip	0.335	0	0.405
wuthering	0	0	0.588

$$\cos(\text{SaS}, \text{PaP}) \approx 0.94$$

$$\cos(\text{SaS}, \text{WH}) \approx 0.79$$

$$\cos(\text{PaP}, \text{WH}) \approx 0.69$$

tf-idf weighting has many variants

Term frequency		Document frequency		Normalization	
n (natural)	$tf_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(tf_{t,d})$	t (idf)	$\log \frac{N}{df_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2 + w_2^2 + \dots + w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times tf_{t,d}}{\max_t(tf_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N - df_t}{df_t}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases} 1 & \text{if } tf_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^\alpha$, $\alpha < 1$
L (log ave)	$\frac{1 + \log(tf_{t,d})}{1 + \log(\text{ave}_{t \in d}(tf_{t,d}))}$				

Many search engines allow for different weightings for queries vs. documents

Thank You
